

COL100: Introduction to Computer Science

Lab 8: Strings

February 28, 2025

1 Strings

Strings in Python are used to represent characters and can be defined using single (') or double (") quotes.

Example

```
1 s1 = 'abc'
2 s2 = "123"
```

The literal '123' represents a string of three characters, not the number 123.

1.1 String Operations

You can perform various operations on strings:

1.1.1 Concatenation and Repetition

The '+' operator is used for concatenation, and the '*' operator is used for repetition.

Examples

```
1 # Concatenation
2 result = 'a' + 'a' # Output: 'aa'
3
4 # Repetition
5 repeated = 3 * 'a' # Output: 'aaa'
```

1.1.2 Type Errors

If you try to use '*' on two strings, Python will raise a `TypeError`.

Example

```
1 # This will raise a TypeError
2 error = 'a' * 'a' # Output: TypeError: can't multiply sequence by non-
    int of type 'str'
```

1.2 Type Checking

Python performs type checking to catch errors. For example, in Python 3, the expression `'4' < 3` will raise a `TypeError` because comparing different types does not have a clear meaning.

1.3 String Length and Indexing

You can find the length of a string using the `len()` function and access characters using indexing.

Example

```
1 s = 'abc'
2 length = len(s)          # Output: 3
3 first_char = s[0]        # Output: 'a'
4 last_char = s[-1]        # Output: 'c' (negative indexing)
```

Indexing Errors

Attempting to access an index out of range will raise an `IndexError`.

```
1 # This will raise an IndexError
2 out_of_range = s[3]      # Output: IndexError: string index out of range
```

1.4 Slicing

Slicing allows you to extract substrings from a string.

Example

```
1 substring = s[1:3]       # Output: 'b'
```

If the start index is omitted, it defaults to 0. If the end index is omitted, it defaults to the length of the string.

```
1 full_string = s[:]       # Output: 'abc'
```

Non-contiguous Slicing

You can also slice with a third argument to skip elements.

```
1 sliced = '123456789'[0:8:2] # Output: '1357'
```

1.5 Type Conversions

Python provides built-in functions to convert types, such as `str()`, `int()`, and `float()`.

```
1 num = 30000000
2 fraction = 1 / 2
3 print(num * fraction, 'is', fraction * 100, '%', 'of', num)
4 # Output: 15000000.0 is 50.0 % of 30000000
```

1.6 Using f-strings for Formatting

Introduced in Python 3.6, f-strings provide a way to embed expressions inside string literals.

```
1 print(f'{int(num * fraction)} is {fraction * 100}% of {num}')
2 # Output: 15000000 is 50.0% of 30000000
```

Modifiers

You can include modifiers in f-strings to control the formatting.

```
1 print(f'{num * fraction:,.0f} is {fraction * 100}% of {num:,}')
2 # Output: 15,000,000 is 50.0% of 30,000,000
```

2 Submission Problem

2.1 Palindrome Checker Problem

Problem Statement

You are given a single string s . Your task is to write a program to determine if the string s is a palindrome. A palindrome is a string that reads the same backward as forward, ignoring spaces, punctuation, and case differences.

2.1.1 Instructions

- The first line contains the string s , which may include letters, numbers, spaces, and punctuation.
- Print **True** if the string s is a palindrome; otherwise, print **False**.

Input

```
1 A man, a plan, a canal: Panama
```

Output

```
1 True
```

Explanation: If we ignore spaces, punctuation, and case, the string reads the same forward and backward.

Input

```
1 race a car
```

Output

```
1 False
```

Explanation: Even after ignoring spaces and punctuation, the string does not read the same forward and backward.

Constraints

- The string s will contain only ASCII characters.
- Treat uppercase and lowercase letters as the same for palindrome checking.
- Ignore non-alphanumeric characters.
- Do not use string slicing to reverse the string.

3 Practice Problem

3.1 Finding the Last Position of a Substring

Problem Statement

Given a string s and a substring sub , write a program to find the last occurrence (position) of sub within s . If the substring does not exist in the string, return -1.

- The first line of input contains the main string s .
- The second line of input contains the substring sub to be searched.
- Output the last position (0-based index) of sub in s .

Note: The search should be case-sensitive.

Input

```
1 hellohello
2 lo
```

Output

```
1 8
```

Explanation: The substring "lo" occurs twice in the string "hellohello". The last occurrence starts at index 8.

Input

```
1 openai
2 ai
```

Output

```
1 4
```

Explanation: The substring "ai" occurs only once in "openai", starting at index 4.

Input

```
1 testcase
2 xyz
```

Output

```
1 -1
```

Explanation: The substring "xyz" does not appear in "testcase", so the output is -1.

Constraints

- The length of the main string s will be between 1 and 10^5 .
- The length of the substring sub will be between 1 and 10^4 .

3.2 Character Frequency Counter

Problem Statement

Write a program that counts the occurrences of each character in the string and prints the frequency of each character in alphabetical order.

- The input consists of a single line containing the string s , which may include letters, numbers, spaces, and punctuation.
- Output each unique character from the string s (ignoring case) along with its frequency, in alphabetical order. Treat uppercase and lowercase letters as the same character.
- Ignore spaces in the frequency count.

Input

```
1 Hello , World!
```

Output

```
1 ,: 1
2 !: 1
3 d: 1
4 e: 1
5 h: 1
6 l: 3
7 o: 2
8 r: 1
9 w: 1
```

Explanation: In the string "Hello, World!", the program counts each character's occurrences, ignoring case and spaces, and outputs the frequency of each character in alphabetical order.

Input

```
1 Data Science
```

Output

```
1 a: 2
2 c: 2
3 d: 1
4 e: 2
5 i: 1
6 n: 1
7 s: 1
8 t: 1
```

Explanation: In the string "Data Science", each character frequency is counted and displayed in alphabetical order.

3.3 Minimum Coins Problem

Problem Statement

Given a list of coin denominations and a target amount, write a program to determine the minimum number of coins needed to make up that amount. If the target amount cannot be achieved with any combination of the available coins, return -1.

- The first line contains two integers: the number of coin denominations n and the target amount amount.
- The second line contains n integers representing the available coin denominations.
- Output the minimum number of coins needed to make up the amount, or -1 if it is not possible.

Input

```
1 3 11
2 1 2 5
```

Output

```
1 3
```

Explanation: To achieve a target amount of 11, we can use three coins: two 5-coin denominations and one 1-coin denomination ($5 + 5 + 1 = 11$).

Input

```
1 2 3
2 2 4
```

Output

```
1 -1
```

Explanation: It is impossible to achieve the target amount of 3 with only coins of denominations 2 and 4.

Constraints

- $1 \leq n \leq 100$
- $1 \leq \text{amount} \leq 10^4$
- Each coin denomination will be a positive integer not greater than 10^4 .